

Cedric's Backend Nachschlagewerk – Version 1

Dieses Dokument basiert auf dem Login-System, das du selbst mit PHP, MySQL, Sessions und Passwort-Hashing aufgebaut hast.

Kapitel 1 – Was ist Backend?

Frontend ist das, was der Benutzer sieht. Backend ist alles, was im Hintergrund passiert. Beispiele: Login prüfen, Daten speichern, Datenbanken abfragen, Benutzer registrieren und Sessions verwalten. Beispiel aus unserem Projekt: form.php → welcome.php → Datenbank → Dashboard
Merksatz: Frontend zeigt. Backend entscheidet.

Kapitel 2 – Datenbanken

Warum brauchen wir Datenbanken? Benutzer sollen dauerhaft gespeichert werden. Unsere Tabelle: users - id - username - password
Merksatz: Benutzer gehören in die Datenbank, nicht in den Code.

AUTO_INCREMENT

AUTO_INCREMENT vergibt automatisch die nächste ID. Beispiel: Cedric = 1 Max = 2 Anna = 3
Typischer Anfängerfehler: 'Warum fehlt die ID 4?' Antwort: IDs müssen nicht lückenlos sein.
AUTO_INCREMENT zählt einfach weiter.

Kapitel 3 – SELECT

SELECT holt Daten aus einer Datenbank. SELECT * FROM users = Alle Benutzer holen. SELECT * FROM users WHERE username = 'Cedric' = Nur Cedric holen. Warum WHERE? Beim Login brauchen wir nur einen Benutzer und nicht die komplette Tabelle.

Kapitel 4 – Login-System

Ablauf: Benutzer gibt Daten ein ↓ PHP bekommt die Daten ↓ PHP fragt die Datenbank ↓ Benutzer gefunden? ↓ Passwort prüfen ↓ Session erstellen ↓ Dashboard öffnen
Wichtig: Die Datenbank entscheidet nicht. PHP entscheidet. Die Datenbank liefert nur Daten.

Kapitel 5 – Sessions

Was ist eine Session? Eine Session merkt sich, dass ein Benutzer eingeloggt ist. Warum brauchen wir sie? Ohne Session würde PHP bei jeder neuen Seite fragen: 'Wer bist du überhaupt?'
Beispiel: Login erfolgreich ↓ Session wird erstellt ↓ Dashboard funktioniert ↓ Logout ↓ Session wird zerstört
Merksatz: Login prüft. Session merkt sich. Logout vergisst.

Kapitel 6 – Registrierung

Ablauf: register.php ↓ Benutzer gibt Daten ein ↓ register_process.php ↓ INSERT INTO users ↓
Benutzer wird gespeichert
Ergebnis: Neue Benutzer können sich einloggen, ohne dass PHP-Code geändert werden muss.

Kapitel 7 – Passwort-Hashing

Früher: Cedric | 1234 Problem: Jeder mit Datenbankzugriff könnte das Passwort lesen. Heute: Cedric | \$2y\$10\$... Das Passwort wird nicht gespeichert. Der Hash wird gespeichert.
password_hash() erzeugt den Hash. password_verify() prüft Passwort gegen Hash. Merksatz:

Passwörter werden nicht gespeichert. Sie werden gehasht gespeichert.

Kapitel 8 – Typische Fehler aus unserem Projekt

Fehler 1: register_process.php direkt geöffnet. Ergebnis: Undefined array key. Lösung: Immer über register.php aufrufen. Fehler 2: Nach Hashing mit username UND password in SQL gesucht. Problem: In der Datenbank steht kein Klartext-Passwort mehr. Lösung: Nur nach username suchen und password_verify() verwenden. Fehler 3: OVR-Modus in VS Code aktiviert. Lösung: Taste EINFÜG / INSERT drücken.

Wichtigster Merksatz des Projekts

PHP fragt die Datenbank. Die Datenbank liefert Daten. PHP entscheidet. Die Session merkt sich den Login.